†

UFABC	**Universidade Federal do ABC** **Bacharelado em Ciência da Computação** **Disciplina:** Sistemas Operacionais **Prof.:** VALÉRIO BATISTA **Turma:** DA1 DA2 NA1 NA2 **Sala:** A-108/114-0 **Exame:** SO - Lista7 **Data:** 29-03-2024

Ass.: _____

Estudante: **ID/RA:**

#E580#V79176 - 2024-03-28 - 20:17:45

	1	2	3	4	5	6	7	8	9	10	11	12
A												
B												
C												
D												

Instruções:

(a) Favor repassar suas respostas neste link: https://forms.gle/ueC5YSay4YD81NLDA
(b) Formulário disponível até 5/4/24 às 23:59:59
(c) Somente a 1a. submissão será registrada (re-submissões serão invalidadas)

Questões de Múltipla Escolha:

1. Considere os seguintes conceitos envolvendo dois processos P, Q e uma região crítica R: (1) P e Q acessam R concomitantemente; (2) P não consegue ser concluído enquando Q usa algum recuso, e vice-versa; (3) P quer acessar R uma 2a. vez, mas na 1a. deixou o *turn* para Q sem que este se interesse mais por R. De 1-3, tais conceitos se referem a, respectivamente

A. Corrida, Deadlock e Starvation. B. Starvation, Corrida e Deadlock. C. Deadlock, Starvation e Corrida. D. Corrida, Starvation e Deadlock.

2. Dentre os suportes de hardware que ajudam a evitar as situações de corrida, o único que não se aplica é

A. Interrupção do processo. B. Troca de Conteúdo entre duas variáveis. C. Acesso a recurso disponível na RAM. D. Leitura e Atualização de um valor da RAM.

3. Considere um processo com apenas duas threads, consumidora e produtora, sendo o código desta dado abaixo, na qual `BUFFER_SIZE` e `count` são globais:

```
1  void *insert(void *args){
2   int valor =* (int *) args;
3   while (count==BUFFER_SIZE);
4   ++count;
5   buffer[count]=valor;}
```

As linhas da região crítica, como integridade dos dados, são

A. 3,4. B. Apenas a 4. C. 3,4,5. D. 4,5.

4. Quanto à solução de Peterson para exclusão mútua, a única alternativa errada é

A. O algoritmo garante a exclusão mútua para duas tarefas concorrentes. B. O algoritmo é correto para muitas tarefas concorrentes, pois as variáveis i, j assumem quaisquer valores inteiros. C. Logo antes de tentarem acessar a região crítica, duas tarefas concorrentes nunca vão conflitar quanto a qual delas terá o acesso. D. Nenhuma das tarefas espera indefinidamente para ter acesso à região crítica.

5. O problema Produtor-Consumidor é um exemplo de

A. Starvation, mas não de Deadlock. B. Deadlock, mas não de Starvation. C. Sincronismo perfeito (nem Deadlock, nem Starvation). D. Deadlock e Starvation.

6. Dados os processos P_1 a P_n com região crítica RC, considere os seguintes conceitos: (1) Depois que P_1 solicita para entrar na RC, há um limite de vezes em que P_2 a P_n entram ali antes de P_1; (2) Se P_1 executa na RC,

então ele é o único ali (P_2 a P_n estão todos excluídos desta RC); (3) Nenhum dentre P_2 a P_n pode bloquear P_1 indefinidamente, se ele solicitar entrar na RC. Os conceitos (1) a (3) referem-se a, respectivamente:

A. Progresso, Espera Limitada e Exclusão Mútua. B. Exclusão Mútua, Progresso e Espera Limitada. C. Exclusão Mútua, Espera Limitada e Progresso. D. Espera Limitada, Exclusão Mútua e Progresso.

7. Com respeito ao mutex, ou semáforo binário S, a única alternativa que não se aplica é

A. A função `release` executa $++S$ e em seguida a tarefa sai da região crítica. B. A função `acquire` testa se $S = 1$, e executa $--S$ em caso afirmativo. C. S começa com valor 0 (i.e., nenhuma tarefa está acessando a região crítica). D. Qualquer tarefa precisa chamar `acquire` para entrar na seção crítica e ter exclusividade ali.

8. A única vantagem em desabilitar interrupções com suporte de hardware é

A. Reduzir este mecanismo a uma única CPU. B. Conceder ao processo a prerrogativa de reabilitá-las quando ele encerrar. C. Não impedir que outras CPUs interfiram no que uma CPU esteja executando. D. Dar a um processo a exclusividade de acessar a memória compartilhada.

9. A exclusão mútua por trava

A. Sempre pode falhar. B. Falha caso não coloquemos um "delay" após lermos o valor da trava. C. Falha caso não coloquemos um "delay" antes de lermos o valor da trava. D. Falha quando há mais de duas tarefas concorrentes.

10. Com respeito ao semáforo contador S com n recursos disponíveis, a única alternativa que não se aplica é

A. A função `acquire` testa se $S > 0$, e executa $--S$ em caso afirmativo. B. A função `release` executa $++S$ e em seguida a tarefa sai da região crítica. C. Caso $S == 0$ e alguma tarefa quiser acessar recurso, então é preciso que uma outra chame `release`. D. S começa com valor 0 (i.e., nenhum recurso está sendo usado).

11. Sejam P e C duas threads de um mesmo processo tais que, em linguagem de máquina, atualizam um contador `count` de ítens num buffer da seguinte forma: `register1 = count`, `register1 = register1 + 1` e `count = register1` (etapas P1, P2 e P3, respec.), `register2 = count`, `register2 = register2 - 1` e `count = register2` (etapas C1, C2 e C3, respec.). Uma vez que as threads rodam em paralelo, se as etapas descritas ocorrerem nos seguntes tempos $t_1 < t_2 < t_3 < t_4$, com t_1 (P1 e C1), t_2 (P2 e C2), t_3 (C3) e t_4 (P3), o valor de `count` será

A. Um a menos que o de início. B. Um a mais que o de início. C. Igual ao valor de início. D. Indefinido.

12. Com respeito à exclusão mútua por chaveamento, a única alternativa falsa é

A. Pode tornar a região crítica inacessível. B. Se for implementada para três ou mais tarefas concorrentes, duas delas podem acessar a região crítica em concomitância. C. Não precisa de "delay" para lermos o valor de *turn* (nem antes, nem depois). D. Não pode ser implementada para mais que duas tarefas concorrentes.